



INSTITUTO POLITÉCNICO NACIONAL ESCUELA SUPERIOR DE CÓMPUTO

PRÁCTICA 5

MAQUINAS DE ESTADOS

DISEÑO DE SISTEMAS DIGITALES

4CM4

INTEGRANTES:

MONTEALEGRE ROSALES DAVID URIEL

SALAUZ HERNANDEZ NERICK FRANCISCO

VÁZQUEZ BLANCAS CESAR SAID

PROFESOR:

AGUILAR SANCHEZ FERNANDO

INSTITUTO POLITÉCNICO NACIONAL



FECHA DE ENTREGA: 24 DE NOVIEMBRE DE 2024

1. OBJETIVO

El alumno verificará el funcionamiento de las Máquinas de Estados Finitos.

2. MATERIAL Y EQUIPO EMPLEADO

- ☐ 1 GAL 22V10
- ☐ 8 resistencias 330 Ω a 1/4 W
- ☐ 8 resistencias 1 K Ω a 1/4 W
- ☐ 8 Leds del color que guste
- ☐ Dip switch de 8
- ☐ Circuito Oscilador con el CI 555 o 74HC14

3. INTRODUCCIÓN TEÓRICA:

Introducción Teórica: Diseño de Sistemas Digitales con Máquinas de Estados

El diseño de sistemas digitales es un área clave en la ingeniería electrónica y computacional que se centra en la creación de circuitos digitales que procesan información de forma secuencial o combinacional. Una de las herramientas fundamentales en este campo es el **modelo de máquinas de estados finitos (FSM, por sus siglas en inglés)**, que proporciona una forma estructurada de modelar el comportamiento de un sistema digital.

Máquinas de Estados Finitos

Una máquina de estados finitos es un modelo matemático que describe un sistema con un número limitado de estados, transiciones entre ellos y salidas asociadas. Este modelo es particularmente útil para sistemas que cambian de estado en respuesta a entradas externas o internas, como controladores, protocolos de comunicación o dispositivos electrónicos secuenciales.

Componentes de una Máquina de Estados

1. **Conjunto de Estados:** Representan las posibles configuraciones del sistema. Cada estado define un momento específico en el tiempo o un comportamiento particular del sistema.
2. **Entradas:** Señales o eventos que afectan al sistema y provocan cambios de estado.
3. **Transiciones:** Reglas que determinan el cambio de un estado a otro en función de las entradas.
4. **Salidas:** Acciones o valores producidos por la máquina de estados como respuesta a las entradas y el estado actual.
5. **Estado Inicial:** El estado en el que comienza el sistema cuando se pone en marcha.

Tipos de Máquinas de Estados

Existen dos tipos principales de máquinas de estados:

- **Máquina de Mealy:** Las salidas dependen del estado actual y de las entradas.
- **Máquina de Moore:** Las salidas dependen únicamente del estado actual.

Proceso de Diseño de una Máquina de Estados

1. **Definición del Problema:** Identificar los requisitos funcionales del sistema.
2. **Identificación de Estados:** Determinar los estados necesarios para modelar el sistema.
3. **Diagrama de Estados:** Dibujar un diagrama que represente los estados y las transiciones.
4. **Tabla de Transiciones:** Crear una tabla que detalle las transiciones y las salidas asociadas.
5. **Codificación de Estados:** Asignar valores binarios a los estados para su implementación.
6. **Implementación:** Usar herramientas como lógica combinacional, flip-flops o microcontroladores para construir físicamente la máquina.

Aplicaciones de las Máquinas de Estados

Las máquinas de estados son fundamentales en el diseño de sistemas como:

- Controladores de tráfico.
- Procesadores de señales digitales.
- Protocolos de comunicación (por ejemplo, UART, I2C, SPI).
- Controladores para robots y autómatas.

Ventajas

- Facilita la comprensión del comportamiento del sistema.
- Simplifica el diseño y la depuración.
- Es escalable para sistemas más complejos.

El diseño de sistemas digitales con máquinas de estados finitos es una metodología poderosa que permite modelar y controlar el comportamiento de sistemas secuenciales de manera eficiente. Dominar este enfoque es esencial para desarrollar soluciones robustas y confiables en ingeniería digital.

1.- Resuelva la siguiente Maquina de Estados que detecta la secuencia 1-1-1 consecutivos sin traslape. Utilice el siguiente diagrama de Estados.

Estado Actual	X	0	1
d0		$d_0/0$	$d_1/0$
d1		$d_0/0$	$d_2/0$
d2		$d_0/0$	$d_3/0$
d3		$d_0/0$	$d_4/1$
d4		$d_0/0$	$d_1/0$
Estado siguiente/Z			

Estado	Q2	Q1	Q0
d0	0	0	0
d1	0	0	1
d2	0	1	0
d3	0	1	1
d4	1	0	0

[illegible]

Tabla de Excitación del FF JK

Q^n	Q^{n+1}	J_0	K_0
0	0	0	*
0	1	1	*
1	0	*	1
1	1	*	0

$Q_2 Q_1 \backslash Q_0 X$	00	01	11	10
00	0 ₀	0 ₁	0 ₃	0 ₂
01	0 ₄	0 ₅	1 ₇	0 ₆
11	* ₁₂	* ₁₃	* ₁₅	* ₁₄
10	0 ₈	0 ₉	* ₁₁	* ₁₀

$$J_2 = Q_1 Q_0 X$$

$Q_2 Q_1 \backslash Q_0 X$	00	01	11	10
00	0 ₀	0 ₁	0 ₃	0 ₂
01	0 ₄	0 ₅	1 ₇	0 ₆
11	* ₁₂	* ₁₃	* ₁₅	* ₁₄
10	* ₈	* ₉	* ₁₁	* ₁₀

$$J_2 = Q_1 Q_0 X$$

$Q_2 Q_1 \backslash Q_0 X$	00	01	11	10
00	* ₀	* ₁	* ₃	* ₂
01	* ₄	* ₅	* ₇	* ₆
11	* ₁₂	* ₁₃	* ₁₅	* ₁₄
10	1 ₈	1 ₉	* ₁₁	* ₁₀

$$K_2 = 1$$

$Q_2 Q_1 \backslash Q_0 X$	00	01	11	10
00	0 ₀	0 ₁	1 ₃	0 ₂
01	* ₄	* ₅	* ₇	* ₆
11	* ₁₂	* ₁₃	* ₁₅	* ₁₄
10	0 ₈	0 ₉	* ₁₁	* ₁₀

$$J_1 = Q_0 X$$

$Q_2Q_1 \backslash Q_0X$	00	01	11	10
00	*	*	*	*
01	1	0	1	1
11	*	*	*	*
10	*	*	*	*

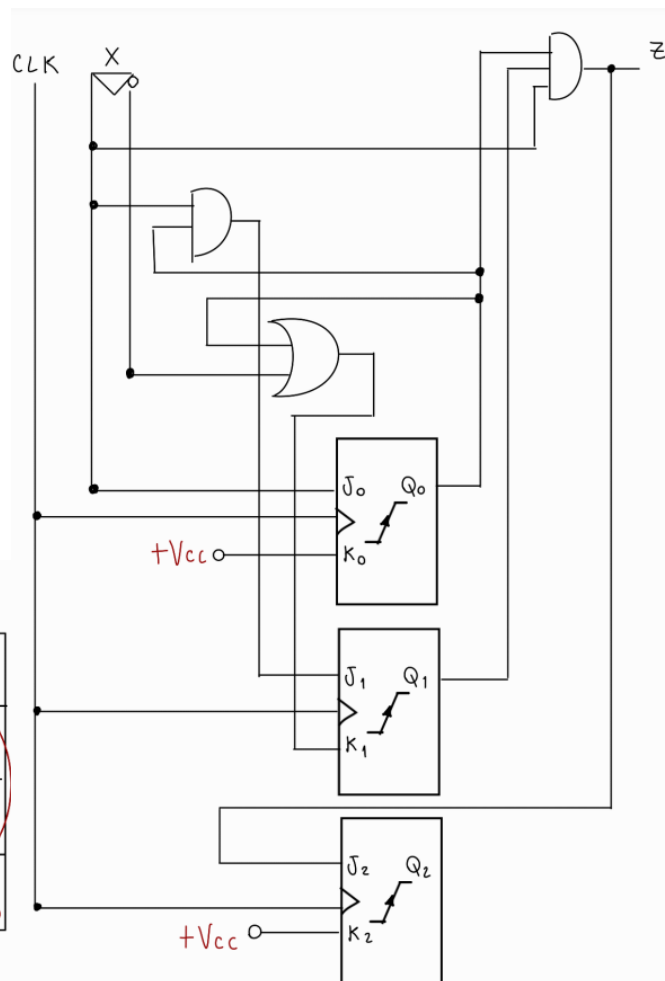
$$K_1 = Q_0 + \bar{X}$$

$Q_2Q_1 \backslash Q_0X$	00	01	11	10
00	0	1	*	*
01	0	1	*	*
11	*	*	*	*
10	0	1	*	*

$$J_0 = X$$

$Q_2Q_1 \backslash Q_0X$	00	01	11	10
00	*	*	1	1
01	*	*	1	1
11	*	*	*	*
10	*	*	*	*

$$K_0 = 1$$



```

library ieee;
use ieee.std_logic_1164.all;
entity diagrama is
    port (clk, x: in std_logic;
          z: out std_logic);
end diagrama;
architecture arq_diagrama of diagrama is
    type estados is (do, d1, d2, d3, d4);
    signal edo_presente, edo_futuro: estados;
begin
    proceso1: process (edo_presente, x) begin
        case edo_presente is
            when do =>
                if x = '1' then
                    edo_futuro <= d1;
                    z <= '0';
                else
                    edo_futuro <= do;
                    z <= '0';
                end if;
            when d1 =>
                if x = '1' then
                    edo_futuro <= d2;
                    z <= '0';
                else
                    edo_futuro <= do;
                    z <= '0';
                end if;
            when d2 =>
                if x = '1' then

```

```

edo_futuro <= d3;

    z <= '0';

else

    edo_futuro <= do;

    z <= '0';

end if;

when d3 =>

    if x = '1' then

        edo_futuro <= d4;

        z <= '1';

    else

        edo_futuro <= do;

        z <= '0';

    end if;

when d4 =>

    if x = '1' then

        edo_futuro <= d1;

        z <= '0';

    else

        edo_futuro <= do;

        z <= '0';

    end if;

end case;

end process;

proceso2: process (clk) begin

    if (clk'event and clk = '1') then

        edo_presente <= edo_futuro;

    end if;

end process;

end arq_diagrama;

```


CODIGO SEGUNDA GAL:

```
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY P6 IS
PORT (E: IN STD_LOGIC;
      DISPLAY: OUT STD_LOGIC_VECTOR ( 6 DOWNT0 0)
);
END ENTITY;
ARCHITECTURE A_P6 OF P6 IS
CONSTANT DIGA: STD_LOGIC_VECTOR ( 6 DOWNT0 0) := "0001000";
CONSTANT DIGE: STD_LOGIC_VECTOR ( 6 DOWNT0 0) := "0110000";
BEGIN
PROCESS (E)
BEGIN
CASE E IS
WHEN "0" => DISPLAY <=DIGE;
WHEN OTHERS => DISPLAY <=DIGA;
END CASE;
END PROCESS;
END A_P6;
```

PINES PRIMERA GAL:

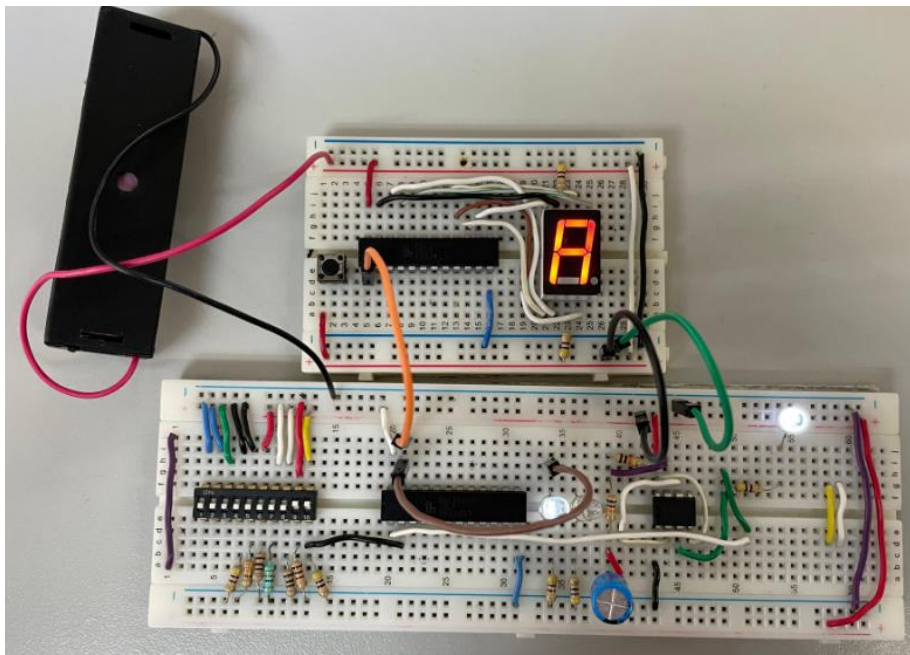
```
C22V10

clk =| 1|                                     |24| * not used
x =| 2|                                     |23|= z
not used *| 3|                             |22| * not used
not used *| 4|                             |21| * not used
not used *| 5|                             |20| * not used
not used *| 6|                             |19| * not used
not used *| 7|                             |18| * not used
not used *| 8|                             |17| * not used
not used *| 9|                             |16| * not used
not used *|10|                             |15|= (edo_present..
not used *|11|                             |14|= (edo_present..
not used *|12|                             |13| * not used
```

PINES SEGUNDA GAL:

```
C22V10

e =| 1|                                     |24| * not used
not used *| 2|                             |23|= display(5)
not used *| 3|                             |22|= display(3)
not used *| 4|                             |21|= display(1)
not used *| 5|                             |20| * not used
not used *| 6|                             |19| * not used
not used *| 7|                             |18| * not used
not used *| 8|                             |17|= display(0)
not used *| 9|                             |16|= display(2)
not used *|10|                             |15|= display(4)
not used *|11|                             |14|= display(6)
not used *|12|                             |13| * not used
```



OBSERVACIONES Y CONCLUSIONES

Montealegre Rosales David Uriel

La práctica permitió observar cómo las máquinas de estados son herramientas fundamentales en el diseño de sistemas digitales secuenciales. La implementación realizada validó correctamente la secuencia 1-1-1-1 mediante un conjunto bien definido de estados (d0 a d4) y transiciones que dependían de entradas específicas, mostrando un diseño determinista y controlado. La separación de procesos para la lógica combinacional (determinación del estado futuro y salidas) y la lógica secuencial (actualización del estado presente sincronizada con el reloj) evidencia una buena práctica en el diseño en VHDL, garantizando un comportamiento estable y predecible. Sin embargo, una debilidad notable es la falta de un manejo explícito de errores o condiciones inesperadas, lo que puede provocar problemas en aplicaciones más complejas. La retroalimentación en el display, mediante caracteres "A" y "E", es básica pero efectiva; no obstante, su funcionalidad puede ampliarse para representar otros estados o proporcionar mensajes más detallados. Estas observaciones sugieren que el diseño, aunque funcional y robusto, podría mejorarse para ser más escalable y adaptable a escenarios más exigentes.

Vázquez Blancas Cesar Said

La implementación de las máquinas de estados durante la práctica demuestra una estructura clara para controlar secuencias específicas, como la validación de la secuencia 1-1-1-1 y la activación de la salida en el estado final (d4). Las transiciones entre estados se manejaron correctamente con sentencias CASE en VHDL, pero el diseño podría optimizarse. Por ejemplo, en el primer código, donde solo existen dos posibles salidas (DIGA y DIGE), se podría reemplazar el bloque CASE con una sentencia más compacta, como IF-ELSE, mejorando la legibilidad y reduciendo la redundancia. Además, el sistema no contempla casos fuera del rango esperado, lo que representa un riesgo en sistemas más complejos donde podrían surgir entradas inválidas. Por otro lado, la salida al display es funcional, pero su alcance es limitado; utilizar representaciones dinámicas o agregar un tercer estado intermedio para indicar procesos en curso podría mejorar la interacción con el usuario. Finalmente, incorporar técnicas como la parametrización de estados o la abstracción mediante generics en VHDL facilitaría la escalabilidad del diseño, haciéndolo más adaptable a aplicaciones futuras.

Salauz Hernández Nerick Francisco

La práctica demostró cómo las máquinas de estados pueden ser empleadas para diseñar sistemas digitales que respondan adecuadamente a entradas específicas, como la validación de la secuencia 1-1-1-1. La lógica de transición implementada en el segundo código, donde cada estado (d0 a d4) tiene reglas específicas para avanzar al siguiente, garantiza un diseño controlado y predecible. Además, la separación de procesos para manejar la lógica combinacional y la lógica secuencial es un ejemplo de buenas prácticas en VHDL, asegurando que las transiciones de estado se realicen de forma sincronizada con el reloj. Sin

embargo, la retroalimentación proporcionada al usuario mediante un display con los caracteres "A" y "E" puede ser limitada para aplicaciones más complejas. Ampliar esta funcionalidad para incluir indicadores como mensajes de progreso, reset o estados de error más específicos mejoraría la experiencia del usuario. Desde una perspectiva técnica, el diseño actual carece de un mecanismo robusto para manejar condiciones no previstas, lo que puede dificultar la depuración y el mantenimiento del sistema. Incorporar un estado de error global o excepciones manejadas permitiría un diseño más robusto. Adicionalmente, al parametrizar las transiciones de estado o emplear registros de desplazamiento como alternativa, el diseño podría adaptarse con mayor facilidad a sistemas de mayor complejidad, aumentando su flexibilidad y utilidad.

5. BIBLIOGRAFIA

- **Brown, S., & Vranesic, Z. (2008).** *Fundamentals of Digital Logic with VHDL Design* (3rd ed.). McGraw-Hill.
- **Katz, R. H., & Borriello, G. (2014).** *Contemporary Logic Design* (2nd ed.). Pearson.
- **Mano, M. M., & Ciletti, M. D. (2013).** *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog* (5th ed.). Pearson.
- **Pedroni, V. A. (2004).** *Circuit Design with VHDL*. MIT Press.